```
// RPMX game server — accounts, leaderboards, friends, clans, chat, online rooms.
// Run: npm install && npm start   (listens on $PORT, default 8080)
const http = require('http');
const fs = require('fs');
const path = require('path');
const crypto = require('crypto');
const { WebSocketServer } = require('ws');

const PORT = process.env.PORT || 8080;
const DATA = process.env.DATA_FILE || path.join(__dirname, 'data.json');
const MAX_ROOM = 12;

// ---------------- storage ----------------
let db = { users: {}, byEmail: {}, clans: {}, chat: [], seq: 1 };
try { db = Object.assign(db, JSON.parse(fs.readFileSync(DATA, 'utf8'))); } catch
let dirty = false;
const save = () => { dirty = true; };
setInterval(() => { if (!dirty) return; dirty = false; fs.writeFile(DATA + '.tmp'

const weekId = () => { const d = new Date(); const t = Date.UTC(d.getUTCFullYear(
const clean = (s, n) => String(s || '').replace(/[<>]/g, '').trim().slice(0, n);
const pubUser = u => u && ({ id: u.id, name: u.name, level: u.p.level || 1, best:
const hashPw = (pw, salt) => crypto.scryptSync(String(pw), salt, 32).toString('he

// ---------------- live state ----------------
const online = {};          // userId -> socket
const rooms = {};           // code -> {code, name, map, host, priv, players: Map
const roomOf = {};          // userId -> code

function send(ws, t, d, rid, err) { if (ws && ws.readyState === 1) ws.send(JSON.s
function toUser(id, t, d) { send(online[id], t, d); }
function roomInfo(r) { return { code: r.code, name: r.name, map: r.map, priv: r.p
function roomBroadcast(r, t, d, except) { for (const id of r.players.keys()) if (
function leaveRoom(uid) {
  const code = roomOf[uid]; if (!code) return; delete roomOf[uid];
  const r = rooms[code]; if (!r) return;
  r.players.delete(uid);
  if (!r.players.size) delete rooms[code];
  else { if (r.host === uid) r.host = r.players.keys().next().value; roomBroadcas
}
function newCode() { let c; do { c = String(Math.floor(100000 + Math.random() * 9
function joinRoom(u, r, d) {
  if (r.players.size >= MAX_ROOM && !r.players.has(u.id)) throw new Error('full')
```

```
  leaveRoom(u.id);
  r.players.set(u.id, { id: u.id, name: u.name, car: clean(d.car, 20), look: d.lo
  roomOf[u.id] = r.code; r.states = r.states || {};
  roomBroadcast(r, 'room', roomInfo(r));
  return roomInfo(r);
}

// live state relay at 12 Hz
setInterval(() => {
  for (const r of Object.values(rooms)) {
    if (!r.states) continue;
    const list = Object.values(r.states); if (!list.length) continue;
    r.states = {};
    roomBroadcast(r, 'st', list);
  }
}, 83);

// ---------------- handlers ----------------
const H = {
  hello(ws, d) {
    let u = d.token && Object.values(db.users).find(x => x.token === d.token);
    if (!u) { const id = db.seq++; u = db.users[id] = { id, token: crypto.randomB
    ws.uid = u.id; if (online[u.id] && online[u.id] !== ws) try { online[u.id].cl
    online[u.id] = ws;
    if (d.name) u.name = clean(d.name, 16) || u.name;
    H.profile(ws, d.profile || {});
    save();
    return { token: u.token, me: Object.assign(pubUser(u), { email: u.email || nu
  },
  register(ws, d, u) {
    const em = clean(d.email, 80).toLowerCase(); if (!/^[^@\s]+@[^@\s]+\.[^@\s]+$
    if (String(d.pass || '').length < 6) throw new Error('bad-pass');
    if (db.byEmail[em] && db.byEmail[em] !== u.id) throw new Error('exists');
    const salt = crypto.randomBytes(12).toString('hex'); u.email = em; u.salt = s
    return { ok: true };
  },
  login(ws, d) {
    const em = clean(d.email, 80).toLowerCase(), id = db.byEmail[em], u = id && d
    if (!u || hashPw(d.pass, u.salt) !== u.pw) throw new Error('wrong');
    ws.uid = u.id; online[u.id] = ws;
    return { token: u.token, me: Object.assign(pubUser(u), { email: u.email, frie
  },
  profile(ws, p, u) {
    u = u || db.users[ws.uid]; if (!u) return;
    const w = weekId(); if (u.week !== w) { u.week = w; u.p.weekly = 0; }
    for (const k of ['level', 'best', 'total', 'weekly', 'cars', 'donuts', 'dist'
```

```
  if (p.car) u.p.car = clean(p.car, 20);
  if (u.clan && db.clans[u.clan]) { const c = db.clans[u.clan]; c.week = c.week
  save(); return { ok: true };
},
rename(ws, d, u) { const n = clean(d.name, 16); if (n.length < 3) throw new Err
leaders(ws, d) {
  const k = d.kind === 'week' ? 'weekly' : d.kind === 'level' ? 'level' : 'best
  const all = Object.values(db.users).map(pubUser).filter(x => x[k] > 0).sort((
  const meIdx = all.findIndex(x => x.id === ws.uid);
  return { top: all.slice(0, 50), me: meIdx >= 0 ? { rank: meIdx + 1, ...all[me
},
// ----- friends & inbox -----
find(ws, d) { const q = clean(d.q, 16).toLowerCase(); if (!q) return []; return
friendAdd(ws, d, u) {
  const f = db.users[d.id]; if (!f || f.id === u.id) throw new Error('no-user')
  if (u.friends.includes(f.id)) return { ok: true };
  f.inbox.push({ k: 'freq', from: u.id, n: u.name, t: Date.now() }); toUser(f.i
},
friendAccept(ws, d, u) {
  const f = db.users[d.id]; if (!f) throw new Error('no-user');
  if (!u.friends.includes(f.id)) u.friends.push(f.id); if (!f.friends.includes(
  u.inbox = u.inbox.filter(m => !(m.k === 'freq' && m.from === f.id)); toUser(f
},
friendRemove(ws, d, u) { u.friends = u.friends.filter(x => x !== d.id); const f
friends(ws, d, u) { return u.friends.map(id => pubUser(db.users[id])).filter(Bo
inbox(ws, d, u) { return u.inbox.slice(-50).reverse(); },
dm(ws, d, u) { const f = db.users[d.id]; if (!f) throw new Error('no-user'); co
// ----- chat -----
chat(ws, d, u) {
  const m = { n: u.name, m: clean(d.m, 160), t: Date.now(), c: d.clan ? u.clan
  if (!m.m) return { ok: false };
  if (m.c) { const c = db.clans[m.c]; if (!c) throw new Error('no-clan'); c.cha
  else { db.chat.push(m); db.chat = db.chat.slice(-100); for (const id in onlin
  save(); return { ok: true };
},
chatLog(ws, d, u) { if (d.clan) { const c = db.clans[u.clan]; return c ? (c.cha
// ----- clans -----
clanCreate(ws, d, u) {
  const tag = clean(d.tag, 5).toUpperCase(), name = clean(d.name, 20);
  if (tag.length < 2 || name.length < 3) throw new Error('bad-clan'); if (db.cl
  db.clans[tag] = { tag, name, icon: clean(d.icon, 10) || 'crown', owner: u.id,
},
clanJoin(ws, d, u) { const c = db.clans[clean(d.tag, 5).toUpperCase()]; if (!c)
clanLeave(ws, d, u) { const c = db.clans[u.clan]; if (c) { c.members = c.member
clanInfo(ws, d, u) { const c = db.clans[d.tag || u.clan]; if (!c) return null;
clans(ws, d) {
```

```
    const w = weekId();
    return Object.values(db.clans).map(c => { const ms = c.members.map(id => db.u
      return { tag: c.tag, name: c.name, icon: c.icon, n: ms.length, level: clanL
      .sort((a, b) => (d.kind === 'week' ? b.week - a.week : b.total - a.total)).
  },
  // ----- rooms -----
  rooms(ws, d) { return Object.values(rooms).filter(r => !r.priv).map(roomInfo).s
  create(ws, d, u) { const code = newCode(); const r = rooms[code] = { code, name
  join(ws, d, u) {
    let r = d.code ? rooms[clean(d.code, 6)] : null;
    if (d.code && !r) throw new Error('no-room');
    if (!r) r = Object.values(rooms).filter(x => !x.priv && x.players.size < MAX_
    if (!r) return H.create(ws, Object.assign({}, d, { rname: 'RPMX ' + (Object.k
    return joinRoom(u, r, d);
  },
  leave(ws, d, u) { leaveRoom(u.id); return { ok: true }; },
  st(ws, d, u) { const r = rooms[roomOf[u.id]]; if (!r) return; d.i = u.id; r.sta
};
function clanLevel(c) { const ms = c.members.map(id => db.users[id]).filter(Boole

// ---------------- server ----------------
const srv = http.createServer((req, res) => {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
  res.end(JSON.stringify({ ok: true, game: 'RPMX', players: Object.keys(online).l
});
const wss = new WebSocketServer({ server: srv, maxPayload: 64 * 1024 });
wss.on('connection', ws => {
  ws.alive = true; ws.on('pong', () => { ws.alive = true; });
  let n = 0, t0 = Date.now();
  ws.on('message', raw => {
    if (Date.now() - t0 > 1000) { t0 = Date.now(); n = 0; } if (++n > 60) return;
    let m; try { m = JSON.parse(raw); } catch (e) { return; }
    const h = H[m.t]; if (!h) return send(ws, null, null, m.rid, 'unknown');
    const u = db.users[ws.uid];
    if (m.t !== 'hello' && m.t !== 'login' && !u) return send(ws, null, null, m.r
    try { const out = h(ws, m.d || {}, u); if (m.rid) send(ws, 'r', out, m.rid);
    catch (e) { if (m.rid) send(ws, null, null, m.rid, e.message || 'error'); }
  });
  ws.on('close', () => { if (ws.uid && online[ws.uid] === ws) { delete online[ws.
});
setInterval(() => { for (const ws of wss.clients) { if (!ws.alive) { ws.terminate
srv.listen(PORT, () => console.log('RPMX server on port', PORT));
```